

# Towards Statically Reasoning about R Vectors

Manuel Di Agostino

University of Parma  
Parma, Italy  
manuel.diagostino@studenti.unipr.it

Florian Sihler

Ulm University  
Ulm, Germany  
florian.sihler@uni-ulm.de

Vincenzo Arceri

University of Parma  
Parma, Italy  
vincenzo.arceri@unipr.it

Oliver Gerstl

Ulm University  
Ulm, Germany  
oliver.gerstl@uni-ulm.de

Matthias Tichy

Ulm University  
Ulm, Germany  
matthias.tichy@uni-ulm.de

## Abstract

R is a dynamically typed, vector-oriented language widely used for data analysis. Its vector semantics, including automatic type coercion, recycling, and flexible selection, are pervasive, non-trivial, and deeply intertwined with the semantics of virtually every R operation. The complexity and counter-intuitive nature of this semantics make static reasoning about R programs particularly challenging, and existing tools offer only shallow analyses that fall short of reasoning about R vector-manipulating programs.

In this paper, we present a parametric abstract domain for R vectors, defined over  $\mu R$ , a core calculus we design to capture the important vector operations in R. An abstract vector simultaneously captures the possible lengths of the vector, the values of its elements, and its potential attributes. The domain is parametric in the element abstract domain. We define abstract operators for all  $\mu R$  operations and equip the domain with a widening operator to ensure termination. We implement the domain on top of flowR, a static analysis framework for R, and evaluate it on a suite of 61 handcrafted programs covering six categories of vector operations, demonstrating the feasibility and precision of the approach.

**CCS Concepts:** • Software and its engineering → Automated static analysis; • Theory of computation → Program analysis.

**Keywords:** Abstract interpretation, Static analysis, R, Vectors, Abstract domains

## ACM Reference Format:

Manuel Di Agostino, Florian Sihler, Vincenzo Arceri, Oliver Gerstl, and Matthias Tichy. 2026. Towards Statically Reasoning about R Vectors. In *Proceedings of the 11th ACM SIGPLAN International Workshop on Numerical and Symbolic Abstract Domains (NSAD '26)*.



This work is licensed under a Creative Commons Attribution 4.0 International License.

NSAD '26, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2933-1/2026/10

<https://doi.org/10.1145/3840563.3843708>

October 4–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3840563.3843708>

## 1 Introduction

R is a dynamically typed, interpreted language originally designed for statistical computing [14]. It is now widely adopted in data science and social science, backed by a rich ecosystem and a large community of researchers and practitioners. R's dynamic nature makes programs notoriously difficult to analyze statically [12, 17]. The language combines dynamic typing, lazy evaluation with observable side effects, first-class environments, reflective meta-programming, and copy-on-modify value semantics. Variables are untyped and may change both their type and structure at any point.

At the heart of R's challenges for static analyzers lies its vector-centric data model. Unlike most languages, R has no scalar type and treats essentially every value as an atomic vector. For instance, constants like TRUE or 3.14 are vectors of length one. Higher-level structures such as matrices or dataframes are vectors with special attributes. Beyond their ubiquity, vectors have first-order, non-trivial, data-sensitive, and often counter-intuitive semantics, such as vector recycling, which repeats the shorter vector in a vectorized operation until it matches the other's length. For these reasons, any static analysis for R must have a sound and precise vector abstraction.

In this paper, we present a parametric abstract domain for R vectors. To isolate the challenges of R's vector semantics, we first define a small but expressive core calculus for R, called  $\mu R$ , that captures the important vector manipulation operations in the language (Section 2). We then define our abstract domain on top of  $\mu R$ , tracking a vector's length, the values of its individual elements up to a statically determined sequence, a summary for the remaining positions, and its possible attributes (Section 3). The domain is parametric in the element abstract domain: any domain providing arithmetic, comparison, and logical operations over scalar values can be plugged in, making the construction reusable across different contexts. We further implement a proof-of-concept analysis on top of flowR [17], using its control-flow graph as the program representation, instantiate our domain with

an interval domain for numeric values, and conduct an empirical evaluation on a suite of systematically handcrafted  $\mu R$  programs that cover all formalized features (Section 4). For each test case, we verify that the abstract result soundly over-approximates the concrete R semantics and remains precise enough.

**Related Work.** Static analysis support for R is limited: flowR [17] derives dataflow graphs for slicing, call-graph analysis, and linting, informed by empirical studies of common R idioms [16], but, like other approaches to R [10, 15], does not construct an abstract domain for R’s vectors and hence fails to compute most value-level properties.

Abstract interpretation has instead been applied more extensively to other dynamic languages: TAJs and JSAI [8, 9] model JavaScript’s prototype chains and dynamically created properties, and Monat et al. [11] develop abstract domains for Python’s lists and dictionaries within the Mopsa framework. These works, however, target heterogeneous, string-indexed objects and dictionaries, unlike R’s homogeneous vectors.

Closer to our setting, domains for dynamically-sized collections include the FunArray parametric segmentation functor [4], which partitions arrays into segments with uniform abstractions, and the sequence domain of Giet et al. [6], which tracks length, extremal values, and sortedness. FunArray’s uniform segments carry no per-position values, yet R’s recycling re-indexes the shorter operand. To apply recycling, FunArray should unroll every position into a singleton segment, defeating segmentation, or summarize and lose exact per-position facts. Giet et al.’s domain is ill-suited for a similar reason: it tracks whole-sequence properties rather than per-position values, yet recycling needs per-position values and does not preserve sortedness. Our domain tracks known content position-by-position and absorbs the remaining elements, if any, in the summary, recovering such facts exactly. Most directly relevant, [13] and [5] model dataframe transformations. However, none of these approaches address R’s vector semantics, which motivates the domain presented in Section 3.

## 2 The $\mu R$ Core Calculus

### 2.1 R Vector Behaviors

R is a vector-oriented language. Atomic vectors are the core runtime values, and even individual Booleans, integers, and floating-point constants are vectors of length one, while NULL denotes the empty vector of length zero [18]. The vector constructor `c()` assembles its arguments into a single vector, integer literals carry an `L` suffix (e.g., `2L`), and vectors are 1-indexed. Consider the following simple R program:

```
1 x <- c(TRUE, 2L, 3.0)
2 y <- x + c(10, 20)
3 z <- y[c(TRUE, FALSE, NA)]
4 z[c(2, 4)] <- c(50, 100)
```

**Type coercion.** R enforces type homogeneity within atomic vectors via implicit coercion, promoting elements to the least upper bound type under `BOOL < INT < FLOAT`. At line 1, `TRUE` and `2L`, an R integer literal, are silently promoted to `FLOAT`, yielding `[1.0, 2.0, 3.0]`. **Recycling.** Vectorized operations cyclically repeat their shorter operands to match the longest one, a mechanism called *recycling*. In our example, line 2 adds a length-2 vector to a length-3 vector: `c(10, 20)` is recycled to `[10, 20, 10]`, producing `[11.0, 22.0, 13.0]`. **Missing values and attributes.** NA propagates through operations (line 3: NA in the mask yields `[11.0, NA]`, and after line 4, `z` is `[11.0, 50.0, NA, 100.0]`). Beyond data, vectors can carry named *attributes* (e.g., `names`, `dim`, `class`) that govern R’s S3 function dispatch [18] and how vectors are interpreted and interact. For brevity, we focus on default methods for atomic vectors without attributes that might override the semantics of the operators we model. We do, however, track the presence of attributes in our abstract domain (Section 3).

### 2.2 Concrete Semantics

We define  $\mu R$ , a core calculus that captures R’s fundamental vector operations while abstracting away environments, promises, and lazy evaluation. We omit  $\mu R$ ’s control-flow constructs (loops and conditionals) here for brevity, as they are orthogonal to the vector-domain analysis that is the focus of this paper. Figure 1 summarizes the syntax. Explicit vector creation occurs via `c()` with zero or more arguments. For subsetting, we focus on `[` to select multiple items and `[[` to extract a single item, which we treat as `[` restricted to a single positive index.  $\mu R$  does not model the `$` operator, since we do not handle name-sensitive access. Binary operations (arithmetic, comparison, logical) are vectorized and coerce to a common type. Attribute manipulation is captured by `setAttr` and `getAttr`.

**Semantic domain.** We model a vector  $v$  as a pair  $(\text{data}, \mathcal{A})$  where  $\text{data} : \mathbb{N}^* \rightarrow \mathbb{V}_{\text{Atom}}$  is a total function mapping positive natural numbers to atomic values, and  $\mathcal{A} \in \wp(\text{String})$  is the set of known attributes. We write  $[c_1, \dots, c_n]_A$  to denote a vector whose data component maps each position  $i \in [1, n]$  to  $c_i$  and all  $i > n$  to `Null`, and whose attribute set is  $A$ . When  $A = \emptyset$  (i.e., the vector has no special attributes of interest) we write  $[c_1, \dots, c_n]$ .

The use of `Null` (not NA) for positions beyond the vector’s length distinguishes out-of-bounds access from legitimate NA elements. The active domain of  $v$  is  $\text{dom}(v) \stackrel{\text{def}}{=} \{i \in \mathbb{N}^* \mid v(i) \neq \text{Null}\}$ , and its length is  $|v| \stackrel{\text{def}}{=} |\text{dom}(v)|$ . The empty vector is `[]`, the constant function returning `Null` everywhere.

Concrete states are partial functions  $\mathcal{E} : \text{Var} \rightarrow \mathcal{V}$ . The denotational semantics has signature  $\mathbb{B}[E] : \mathcal{E} \rightarrow (\mathcal{E} \times (\mathcal{V} \cup \wp(\text{String})))$ . Since  $\mu R$  supports assignments with side effects, the semantics returns a modified state alongside the resulting

|                    |       |                                                                                    |
|--------------------|-------|------------------------------------------------------------------------------------|
| $E$                | $::=$ | $Atm \mid \triangleright E$                                                        |
|                    | $ $   | $E_1 \diamond_{arith} E_2 \mid E_1 \diamond_{com} E_2 \mid E_1 \diamond_{log} E_2$ |
|                    | $ $   | $c() \mid c(E_1, \dots, E_n)$                                                      |
|                    | $ $   | $Var \leftarrow E$                                                                 |
|                    | $ $   | $Var[E_{idx}] \mid Var[[E_{idx}]]$                                                 |
|                    | $ $   | $Var[E_{idx}] \leftarrow E$                                                        |
|                    | $ $   | $Var[[E_{idx}]] \leftarrow E$                                                      |
|                    | $ $   | $setAttr(Var, \{Str_1, \dots, Str_n\})$                                            |
|                    | $ $   | $getAttr(Var)$                                                                     |
| $E_{idx}$          | $::=$ | $\varepsilon \mid E$                                                               |
| $Atm$              | $::=$ | $Var \mid Float \mid Int \mid Bool \mid NULL$                                      |
| $Bool$             | $::=$ | $TRUE \mid FALSE \mid NA$                                                          |
| $\diamond_{arith}$ | $::=$ | $+ \mid - \mid * \mid /$                                                           |
| $\diamond_{com}$   | $::=$ | $> \mid < \mid == \mid != \mid <= \mid >=$                                         |
| $\diamond_{log}$   | $::=$ | $\& \mid \mid$                                                                     |
| $\triangleright$   | $::=$ | $! \mid -$                                                                         |

**Figure 1.** The syntax of  $\mu R$ . It includes production rules for atoms, unary operations, arithmetic, comparison, and logical operations, assignments, vector construction using  $c$ , subsetting, extraction, and attribute manipulation functions.

value, as formalized in Table 1. We now define the auxiliary operators that model R's vector semantics.

**Type coercion.** We define the set of atomic types  $\mathcal{T} = \{\text{Bool}, \text{Int}, \text{Float}\}$  with the total order  $\text{Bool} \prec \text{Int} \prec \text{Float}$ . The type of a vector is obtained via  $\tau : \mathcal{V} \rightarrow \mathcal{T}$ . Operations involving heterogeneous operands implicitly *coerce* elements to their least upper bound type, denoted  $\tau_1 \sqcup \tau_2$  (e.g.,  $\text{Bool} \sqcup \text{Int} = \text{Int}$ ). The function  $\text{Cast} : \mathcal{V} \times \mathcal{T} \rightarrow \mathcal{V}$  applies this coercion. In particular,  $\text{Cast}(v, t)$  casts each element of  $v$  to type  $t$ , and preserves the vector's attributes. We denote by  $\mathcal{V}_{Int}$ ,  $\mathcal{V}_{Bool}$ , and  $\mathcal{V}_{Float}$  the sets of integer, Boolean, and floating-point vectors, respectively.

**Flattening.** The binary operator  $\oplus : \mathcal{V} \times \mathcal{V} \rightarrow \mathcal{V}$  concatenates two vectors while coercing elements to their common type and preserving only the “names” attribute:

$$[\ ] \oplus x = x \oplus [\ ] \stackrel{\text{def}}{=} [x_1, \dots, x_n]_{A \cap \{\text{"names"}\}}$$

$$x \oplus y \stackrel{\text{def}}{=} [x'_1, \dots, x'_m, y'_1, \dots, y'_n]_{(A'_1 \cup A'_2) \cap \{\text{"names"}\}}$$

where  $t = \tau(x) \sqcup \tau(y)$ ,  $x' = [x'_1, \dots, x'_m]_{A'_1} = \text{Cast}(x, t)$ , and  $y' = [y'_1, \dots, y'_n]_{A'_2} = \text{Cast}(y, t)$ . By abuse of notation, we use  $\oplus$  to concatenate data components as well.

**Recycling.** Binary operations and logical selection masks both apply R's recycling mechanism to align vectors of different lengths. The function  $\text{Recycle} : \mathcal{V} \times \mathcal{V} \rightarrow \mathcal{V} \times \mathcal{V}$  determines the target length  $m = \max(|x|, |y|)$  and delegates

data extension to  $\rho_c$ :

$$\text{Recycle}([\ ], x) = \text{Recycle}(x, [\ ]) \stackrel{\text{def}}{=} ([\ ], [\ ])$$

$$\text{Recycle}(x, y) \stackrel{\text{def}}{=} ((\rho_c(x'.\text{data}, m), x'.\mathcal{A}), (\rho_c(y'.\text{data}, m), y'.\mathcal{A}))$$

where  $t = \tau(x) \sqcup \tau(y)$ ,  $x' = \text{Cast}(x, t)$ ,  $y' = \text{Cast}(y, t)$ . The core recycling function  $\rho_c : (\mathbb{N}^* \rightarrow \mathbb{V}_{Atm} \times \mathbb{N}^*) \rightarrow (\mathbb{N}^* \rightarrow \mathbb{V}_{Atm})$  repeats the data component to reach the target length:

$$\rho_c(x, l) \stackrel{\text{def}}{=} \underbrace{[x_1, \dots, x_n] \oplus \dots \oplus [x_1, \dots, x_n]}_{[l/n] \text{ times}} \oplus \underbrace{[x_1, \dots, x_r]}_{l \bmod n \text{ elements}}$$

For example,  $\text{Recycle}([1, 2], [3, 4, 5]) = ([1, 2, 1], [3, 4, 5])$ .

**Selection.** The function  $\text{Select} : \mathcal{V} \times \mathcal{V}_{Int} \rightarrow \mathcal{V}$  models the  $[\ ]$  operator under three index types. In all cases, the return type is the type of the data vector: 1. **Positive indexing** extracts elements at specified positions. Out-of-bounds indices produce NA, and index 0 contributes nothing to the result. The indices themselves may contain NA, which yields NA at the corresponding output position. 2. **Negative indexing** returns the data vector with the indexed positions *excluded*. All indices must be strictly non-positive. 3. **Logical indexing** uses a Boolean mask recycled to the maximum length between the selector and content vectors. Positions where the mask is True are kept, False are dropped, and NA in the mask produces NA in the output. For instance, given  $v = [10, 20, 30, 40]$ :  $\text{Select}(v, [2, 5]) = [20, \text{NA}]$ ,  $\text{Select}(v, [-1, -3]) = [20, 40]$ , and  $\text{Select}(v, [\text{True}, \text{False}]) = [10, 30]$ . The formal subcase definitions are given in the appendix.

**Update.** The function  $\text{Update} : \mathcal{V} \times \mathcal{V}_{Int} \times \mathcal{V} \rightarrow \mathcal{V}$  models assignment to vector elements, based on index type: 1. **Positive indexing** overwrites elements at specified positions. If the assignment positions exceed the original length, the vector is extended and intermediate positions are filled with NA. The replacement values are recycled to match the number of valid (non-zero) indices. 2. **Negative indexing** updates all positions *except* the excluded ones. The length of the target vector is invariant. Replacement values are recycled to cover the number of updated positions. 3. **Logical indexing** uses a Boolean mask recycled to the maximum length between the selector and content vectors. True positions are overwritten. As in the positive case, the vector may be extended if the mask is longer than the original vector. For instance, given  $v = [10, 20, 30]$ :  $\text{Update}(v, [5], [99]) = [10, 20, 30, \text{NA}, 99]$ ,  $\text{Update}(v, [-2], [5]) = [5, 20, 5]$ , and  $\text{Update}(v, [\text{True}, \text{False}], [0]) = [0, 20, 0]$ . The formal subcase definitions are given in the appendix.

**Expression semantics.** Table 1 defines the denotational semantics of  $\mu R$  expressions. We treat  $[\ ]$  like  $[\ ]$  restricted to a single positive index and omit the separate  $[\ ]$  rows.

The helper functions `ToBool`, `ToBool`, and `ToNum` (which map atomic values to Booleans and numbers) are elided for brevity. Their definitions are in the appendix.

Table 1. Concrete semantics of  $\mu R$  expressions.

|                                                                                                                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\mathbb{E}[E] : \mathcal{E} \rightarrow (\mathcal{E} \times (\mathcal{V} \cup \wp(\text{String})))$                                                                                                                                                                                                                      |
| where $x \in \text{Var}$ , $a \in \text{Atm}$ , $e \in E$ , $s \in \text{Str}$                                                                                                                                                                                                                                            |
| $\mathbb{E}[a]\rho \stackrel{\text{def}}{=} \begin{cases} (\rho, \rho(a)) & \text{if } a \in \text{Var} \\ (\rho, [a]) & \text{if } a \in \text{Float} \cup \text{Int} \cup \text{Bool} \\ (\rho, []) & \text{if } a \equiv \text{NULL} \end{cases}$                                                                      |
| $\mathbb{E}[-e]\rho \stackrel{\text{def}}{=} (\rho', [-\text{ToNum}(c_1), \dots, -\text{ToNum}(c_k)])$<br>where $(\rho', [c_1, \dots, c_k]) = \mathbb{E}[e]\rho$                                                                                                                                                          |
| $\mathbb{E}![e]\rho \stackrel{\text{def}}{=} (\rho', [\text{ToBool}(c_1), \dots, \text{ToBool}(c_k)])$<br>where $(\rho', [c_1, \dots, c_k]) = \mathbb{E}[e]\rho$                                                                                                                                                          |
| $\mathbb{E}[e_1 \diamond e_2]\rho \stackrel{\text{def}}{=} (\rho_2, v'_1 \diamond v'_2)$<br>where $(\rho_1, v_1) = \mathbb{E}[e_1]\rho$ , $(\rho_2, v_2) = \mathbb{E}[e_2]\rho_1$<br>$(v'_1, v'_2) = \text{Recycle}(v_1, v_2)$ , $\diamond \in \{\diamond_{\text{arith}}, \diamond_{\text{rel}}, \diamond_{\text{log}}\}$ |
| $\mathbb{E}[x <- e]\rho \stackrel{\text{def}}{=} (\rho' [x \mapsto v], v)$ where $(\rho', v) = \mathbb{E}[e]\rho$                                                                                                                                                                                                         |
| $\mathbb{E}[x[] <- e]\rho \stackrel{\text{def}}{=} (\rho' [x \mapsto v'], v')$<br>where $(\rho', v) = \mathbb{E}[e]\rho$ , $v' = \rho_c(v,  \rho'(x) )$                                                                                                                                                                   |
| $\mathbb{E}[x[e_{\text{idx}}] <- e]\rho_0 \stackrel{\text{def}}{=} (\rho_3, \rho_3(x))$<br>where $(\rho_1, v) = \mathbb{E}[e]\rho_0$ , $(\rho_2, v_{\text{idx}}) = \mathbb{E}[e_{\text{idx}}]\rho_1$ ,<br>$\rho_3 = \rho_2[x \mapsto \text{Update}(\rho_2(x), v_{\text{idx}}, v)]$                                        |
| $\mathbb{E}[c()]\rho \stackrel{\text{def}}{=} (\rho, [])$                                                                                                                                                                                                                                                                 |
| $\mathbb{E}[c(e_1, \dots, e_n)]\rho_0 \stackrel{\text{def}}{=} (\rho_n, v_1 \oplus \dots \oplus v_n)$<br>where $(\rho_i, v_i) = \mathbb{E}[e_i]\rho_{i-1}$ , $1 \leq i \leq n$                                                                                                                                            |
| $\mathbb{E}[x[]]\rho \stackrel{\text{def}}{=} \mathbb{E}[x]\rho$                                                                                                                                                                                                                                                          |
| $\mathbb{E}[x[e_{\text{idx}}]]\rho \stackrel{\text{def}}{=} (\rho', \text{Select}(\rho'(x), v_{\text{idx}}))$<br>where $(\rho', v_{\text{idx}}) = \mathbb{E}[e_{\text{idx}}]\rho$                                                                                                                                         |
| $\mathbb{E}[\text{getAttr}(x)]\rho \stackrel{\text{def}}{=} (\rho, \mathcal{A})$ where $(c, \mathcal{A}) = \mathbb{E}[x]\rho$                                                                                                                                                                                             |
| $\mathbb{E}[\text{setAttr}(x, \{s_1, \dots, s_n\})]\rho \stackrel{\text{def}}{=} (\rho[x \mapsto v'], v')$<br>where $(c, \mathcal{A}) = \mathbb{E}[x]\rho$ , $v' = (c, \{s_1, \dots, s_n\})$                                                                                                                              |

### 3 An Abstract Domain for $\mu R$ Vectors

We present a non-relational value analysis approximating the vector-based semantics of Section 2. The abstract domain is parametric in the element domain and captures vector length, element values, and attributes.

Full formal definitions are given in the appendix; the summaries below elide some cases for brevity. Soundness is proved for the recycling and positive-selection abstractions. The structural results on the order, widening, and concretization are also proved. The remaining operators are validated empirically (Section 4).

#### 3.1 Prerequisites

We consider three underlying lattices. The length domain is a positive integer interval lattice  $\langle \text{Itv}_0, \sqsubseteq_{\text{Itv}_0}, \sqcup_{\text{Itv}_0}, \sqcap_{\text{Itv}_0}, \perp_{\text{Itv}_0}, \top_{\text{Itv}_0} \rangle$ . The content domain  $\langle \mathcal{B}^\#, \sqsubseteq_{\mathcal{B}^\#}, \sqcup_{\mathcal{B}^\#}, \sqcap_{\mathcal{B}^\#}, \perp_{\mathcal{B}^\#}, \top_{\mathcal{B}^\#} \rangle$  is parametric: any abstract domain which approximates  $\wp(\mathbb{V}_{\text{Atm}})$  can be plugged in, provided it supplies arithmetic, comparison, and logical operations. The abstraction function  $\alpha_{\mathcal{B}^\#} : \mathbb{V}_{\text{Atm}} \rightarrow \mathcal{B}^\#$  maps concrete literals to their best abstract representation. The attribute domain (following [2]) is  $\text{Attr}^\# = \{(\text{MustHave}, \text{MayHave}) \in \wp(\text{Attr}) \times \wp(\text{Attr}) \mid \text{MustHave} \subseteq \text{MayHave}\} \cup \{\perp_{\text{Attr}^\#}\}$  where  $\text{Attr} = \{\text{names}, \text{dim}, \text{class}, \text{other}\}$ . The `MustHave` component captures attributes definitely present, and `MayHave` captures those possibly present. The partial order is  $(\text{MustHave}_1, \text{MayHave}_1) \sqsubseteq_{\text{Attr}^\#} (\text{MustHave}_2, \text{MayHave}_2)$  iff  $\text{MustHave}_2 \subseteq \text{MustHave}_1 \wedge \text{MayHave}_1 \subseteq \text{MayHave}_2$ , with  $\perp_{\text{Attr}^\#}$  below all. Key elements are:  $\text{AttrEmpty} \stackrel{\text{def}}{=} (\emptyset, \emptyset)$ ,  $\text{AttrFull} \stackrel{\text{def}}{=} (\text{Attr}, \text{Attr})$  and  $\top_{\text{Attr}^\#} \stackrel{\text{def}}{=} (\emptyset, \text{Attr})$ .

#### 3.2 The Domain

An abstract vector  $v$  is a 4-tuple  $v = ([l, u], \mathbf{p}, s, a)$ , where  $[l, u] \in \text{Itv}_0$  approximates the length,  $\mathbf{p} = \langle p_1 \dots p_k \rangle \in \mathcal{B}^{k*}$  stores known element abstractions for the first  $k$  positions,  $s \in \mathcal{B}^\#$  is a *summary* approximating all positions beyond  $\mathbf{p}$ , and  $a \in \text{Attr}^\#$  approximates the attribute set. When the upper bound is finite, the known content spans the whole length ( $u = |\mathbf{p}|$ ) and the summary is inactive ( $s = \perp_{\mathcal{B}^\#}$ ); only when  $u = +\infty$  does the summary abstract the positions beyond  $\mathbf{p}$ . The raw domain is:

$$\tilde{\mathcal{V}}_{\mathcal{B}}^\# \stackrel{\text{def}}{=} \left\{ ([l, u], \mathbf{p}, s, a) \mid \begin{array}{l} [l, u] \in \text{Itv}_0, \mathbf{p} \in \mathcal{B}^{k*}, \\ s \in \mathcal{B}^\#, a \in \text{Attr}^\#, \\ l \leq |\mathbf{p}| \leq u, \\ (u \neq +\infty \implies s = \perp_{\mathcal{B}^\#} \wedge u = |\mathbf{p}|) \end{array} \right\}.$$

We define an equivalence relation  $\equiv$  on  $\tilde{\mathcal{V}}_{\mathcal{B}}^\#$ :  $v_1 \equiv v_2$  iff they agree on  $l$ ,  $s$ ,  $a$ , and on the known content up to the shorter sequence's length, with all trailing positions of the longer sequence equal to its summary. The abstract domain is the set of equivalence classes:

$$\mathcal{V}_{\mathcal{B}}^\# \stackrel{\text{def}}{=} \{[v]_\equiv \mid v \in \tilde{\mathcal{V}}_{\mathcal{B}}^\#\} \cup \{\perp_{\mathcal{V}_{\mathcal{B}}^\#}, \top_{\mathcal{V}_{\mathcal{B}}^\#}\}.$$

Each class has a minimum representative  $v^\downarrow$ , obtained by removing trailing positions of  $\mathbf{p}$  equal to  $s$  while keeping at least  $l$  positions. By convention,  $v \in \mathcal{V}_{\mathcal{B}}^\#$  denotes the

class represented by  $v^\perp$ . All operators defined below are well-defined on equivalence classes, i.e., independent of the choice of representative.

The lattice is  $\mathcal{D}_{\mathcal{V}_B^\#} = \langle \mathcal{V}_B^\#, \sqsubseteq_{\mathcal{V}_B^\#}, \sqcup_{\mathcal{V}_B^\#}, \sqcap_{\mathcal{V}_B^\#}, \perp_{\mathcal{V}_B^\#}, \top_{\mathcal{V}_B^\#} \rangle$  where  $\perp_{\mathcal{V}_B^\#} \stackrel{\text{def}}{=} (\perp_{\text{Itv}_0}, \varepsilon, \perp_{\mathcal{B}^\#}, \perp_{\text{Attr}^\#})$ ,  $\top_{\mathcal{V}_B^\#} \stackrel{\text{def}}{=} (\top_{\text{Itv}_0}, \varepsilon, \top_{\mathcal{B}^\#}, \top_{\text{Attr}^\#})$ , and  $\varepsilon$  denotes the empty sequence. The abstraction of the empty vector differs from  $\perp_{\mathcal{V}_B^\#}$  in its length and attributes:  $[ ]_{\mathcal{V}_B^\#} \stackrel{\text{def}}{=} ([0, 0], \varepsilon, \perp_{\mathcal{B}^\#}, \text{AttrEmpty})$ . Throughout the paper, we use  $\mathbf{p}$  to denote the entire known content sequence, and  $p_i$  (or  $p_{v,i}$  when referring to a specific vector  $v$ ) to denote the abstract value at position  $i$  within that sequence. When multiple vectors are in scope, we write  $\mathbf{p}_1 = \langle p_{1,1} \dots p_{1,k_1} \rangle$  and  $\mathbf{p}_2 = \langle p_{2,1} \dots p_{2,k_2} \rangle$ .

### 3.3 Lattice Operations

**Partial order.**  $v_1 \sqsubseteq_{\mathcal{V}_B^\#} v_2$  iff  $L_1 \sqsubseteq_{\text{Itv}_0} L_2 \wedge s_1 \sqsubseteq_{\mathcal{B}^\#} s_2 \wedge a_1 \sqsubseteq_{\text{Attr}^\#} a_2 \wedge K(v_1, v_2)$ , where  $K(v_1, v_2)$  requires  $\forall i \in [1, k_1] : p_{1,i} \sqsubseteq_{\mathcal{B}^\#} p_{2,i}$  when  $k_1 \leq k_2$  and  $u_1 \neq +\infty$ . (T1) The relation  $\sqsubseteq_{\mathcal{V}_B^\#}$  is an order relation on  $\mathcal{V}_B^\#$ . (Full case analysis in the appendix.)

**Join.** The join is component-wise:  $L_1 \sqcup_{\text{Itv}_0} L_2$  on length intervals,  $\mathbf{p}_1 \sqcup_{\mathcal{B}^\#} \mathbf{p}_2$  on known content,  $s_1 \sqcup_{\mathcal{B}^\#} s_2$  on summaries, and  $a_1 \sqcup_{\text{Attr}^\#} a_2$  on attributes. The content join  $\mathbf{p}_1 \sqcup_{\mathcal{B}^\#} \mathbf{p}_2 \stackrel{\text{def}}{=} \langle p_{1,i} \sqcup_{\mathcal{B}^\#} p_{2,i} \rangle_{i=1}^m \oplus \text{tail}$  with  $m = \min(k_1, k_2)$ , where tail joins the excess positions of the longer sequence with the shorter's summary. The attribute join is  $a_1 \sqcup_{\text{Attr}^\#} a_2 = (\text{MustHave}_1 \cap \text{MustHave}_2, \text{MayHave}_1 \cup \text{MayHave}_2)$ . (Full case analysis of tail in the appendix.)

**Meet.** Let  $u'$  denote the largest index whose pointwise meet is non- $\perp_{\mathcal{B}^\#}$  (using summaries beyond the known content). We distinguish three cases. If the lower bounds conflict ( $\max(l_1, l_2) > u'$ ), the meet is  $\perp_{\mathcal{V}_B^\#}$ . If both vectors have infinite length and a non- $\perp_{\mathcal{B}^\#}$  summary meet, the result is also infinite. Otherwise, the result is finite. The known content  $\mathbf{p}' = \langle p'_i \rangle_{i=1}^{u'}$  is built positionwise:  $p'_i = p'_{1,i} \sqcap_{\mathcal{B}^\#} p'_{2,i}$  where  $p'_{j,i}$  uses the actual known value or the summary beyond the known length. The attribute meet is  $a_1 \sqcap_{\text{Attr}^\#} a_2 = (\text{MustHave}_1 \cup \text{MustHave}_2, \text{MayHave}_1 \cap \text{MayHave}_2)$  when valid, otherwise  $\perp_{\text{Attr}^\#}$ .

**Widening.**  $v_1 \nabla_{\mathcal{V}_B^\#} v_2 \stackrel{\text{def}}{=} ([l', u'], \mathbf{p}', s', a_1 \sqcup_{\text{Attr}^\#} a_2)$  where  $[l', u'] = L_1 \nabla_{\text{Itv}_0} L_2$ . The content  $\mathbf{p}'$  extends to  $\max(k_1, k_2)$  when the widened length is finite or both  $u_1$  and  $u_2$  are finite. Otherwise, it truncates at  $\min(k_1, k_2)$  positions (value-widened pairwise through  $\nabla_{\mathcal{B}^\#}$ ). The summary  $s' = \perp_{\mathcal{B}^\#}$  when  $u' \neq +\infty$ . Otherwise, it is the value-widening of  $s_1$  with the join of  $s_2$  and all content positions beyond the common length. (T2)  $\nabla_{\mathcal{V}_B^\#}$  is well-founded: it is an upper-bound

operator and ensures termination of widening sequences. (Full case analysis in the appendix.)

**Concretization.** The analysis is *concretization-based*: it relies solely on the concretization  $\gamma_{\mathcal{V}_B^\#}$ , since no vector-level best abstraction exists. Only the element-level abstraction  $\alpha_{\mathcal{B}^\#}$  of Section 3.1 is assumed, with per-element soundness.

$$\gamma_{\mathcal{V}_B^\#}([l, u], \mathbf{p}, s, a) \stackrel{\text{def}}{=} \left\{ [c_1, \dots, c_n]_A \mid \begin{array}{l} n \in \gamma_{\text{Itv}_0}([l, u]), \\ \forall i \leq \min(n, |\mathbf{p}|) : c_i \in \gamma_{\mathcal{B}^\#}(p_i), \\ \forall j > |\mathbf{p}| : c_j \in \gamma_{\mathcal{B}^\#}(s) \\ A \in \gamma_{\text{Attr}^\#}(a) \end{array} \right\}$$

where  $\gamma_{\text{Attr}^\#}(\perp_{\text{Attr}^\#}) = \emptyset$ , and  $\gamma_{\text{Attr}^\#}(\text{MustHave}, \text{MayHave}) = \{A \subseteq \text{Attr} \mid \text{MustHave} \subseteq A \subseteq \text{MayHave}\}$ . (T3) Given a meet-preserving concretization  $\gamma_{\mathcal{B}^\#}$  of the value domain,  $\gamma_{\mathcal{V}_B^\#}$  is meet-preserving:  $\gamma_{\mathcal{V}_B^\#}(v_1 \sqcap_{\mathcal{V}_B^\#} v_2) = \gamma_{\mathcal{V}_B^\#}(v_1) \cap \gamma_{\mathcal{V}_B^\#}(v_2)$ .

### 3.4 Abstract Semantics

All operations defined below assume input vectors have no attributes ( $a = \text{AttrEmpty}$ ). If a non-AttrEmpty attribute component is encountered, the result is conservatively set to  $\top_{\mathcal{V}_B^\#}$ . Abstract states are  $\mathcal{E}^\# : \text{Var} \rightarrow \mathcal{V}_B^\#$  and the abstract expression semantics has signature  $\mathbb{E}[E]^\# : \mathcal{E}^\# \rightarrow \mathcal{E}^\# \times \mathcal{V}_B^\#$ . The core rules are summarized in Table 2.

Table 2. Abstract semantics of  $\mu R$  expressions.

| $\mathbb{E}[E]^\# : \mathcal{E}^\# \rightarrow \mathcal{E}^\# \times \mathcal{V}_B^\#$                                                                                                                                                                                                                                                                         | where $a \in \text{Atm}$ , $e \in E$ |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| $\mathbb{E}[a]^\# \rho \stackrel{\text{def}}{=} \begin{cases} (\rho, \rho(a)) & \text{if } a \in \text{Var} \\ (\rho, ([1, 1], \alpha_{\mathcal{B}^\#}(a), \perp_{\mathcal{B}^\#}, \text{AttrEmpty})) & \text{if } a \text{ literal} \\ (\rho, ([0, 0], \varepsilon, \perp_{\mathcal{B}^\#}, \text{AttrEmpty})) & \text{if } a \equiv \text{NULL} \end{cases}$ |                                      |
| $\mathbb{E}[-e]^\# \rho \stackrel{\text{def}}{=} (\rho', (i, \langle -^\# c_1 \dots -^\# c_n \rangle, -^\# s, \text{AttrEmpty}))$                                                                                                                                                                                                                              |                                      |
| where $(\rho', (i, \langle c_1 \dots c_n \rangle, s, \text{AttrEmpty})) = \mathbb{E}[e]^\# \rho$                                                                                                                                                                                                                                                               |                                      |
| $\mathbb{E}![e]^\# \rho \stackrel{\text{def}}{=} (\rho', (i, \langle \overline{\text{ToBool}}_{\mathcal{B}^\#}^\#(c_1) \dots \overline{\text{ToBool}}_{\mathcal{B}^\#}^\#(c_n) \rangle, \overline{\text{ToBool}}_{\mathcal{B}^\#}^\#(s), \text{AttrEmpty}))$                                                                                                   |                                      |
| where $(\rho', (i, \langle c_1 \dots c_n \rangle, s, \text{AttrEmpty})) = \mathbb{E}[e]^\# \rho$                                                                                                                                                                                                                                                               |                                      |
| $\mathbb{E}[e_1 \diamond e_2]^\# \rho \stackrel{\text{def}}{=} (\rho_2, (L', \langle p_{1,1} \diamond_{\mathcal{B}^\#}^\# p_{2,1} \dots \rangle, s'_1 \diamond_{\mathcal{B}^\#}^\# s'_2, \text{AttrEmpty}))$                                                                                                                                                   |                                      |
| where $(\rho_1, v_1) = \mathbb{E}[e_1]^\# \rho$ , $(\rho_2, v_2) = \mathbb{E}[e_2]^\# \rho_1$ ,<br>$(v'_1, v'_2) = \text{Recycle}^\#(v_1, v_2)$                                                                                                                                                                                                                |                                      |
| $\mathbb{E}[c()]^\# \rho \stackrel{\text{def}}{=} (\rho, ([0, 0], \varepsilon, \perp_{\mathcal{B}^\#}, \text{AttrEmpty}))$                                                                                                                                                                                                                                     |                                      |
| $\mathbb{E}[c(e_1, \dots, e_n)]^\# \rho_0 \stackrel{\text{def}}{=} (\rho_n, v_1 \oplus^\# \dots \oplus^\# v_n)$                                                                                                                                                                                                                                                |                                      |

Continued on next page

Table 2. Abstract semantics of  $\mu R$  expressions. (Cont.)

where  $(\rho_i, v_i) = \mathbb{E}[e_i] \rho_{i-1}^\#$

Unary and binary operators  $(-\# , \diamond^\# , \text{etc.})$  are delegated to the underlying scalar domain.

**Abstract concatenation**  $\oplus^\#$ . Let  $v_1 = ([l_1, u_1], \mathbf{p}_1, s_1, a_1)$  and  $v_2 = ([l_2, u_2], \mathbf{p}_2, s_2, a_2)$ . When  $u_1 \neq +\infty$ , the result is finite:  $L = [l_1 + l_2, u_1 + u_2]$ ,  $\mathbf{p}' = \mathbf{p}_1 \oplus \mathbf{p}_2$ ,  $s = \perp_{\mathcal{B}^\#}$ , and  $a = a_1 \sqcup_{Attr^\#} a_2$ , with each position  $i \in [u_1 + 1, u_1 + k_2]$  join-propagated backward over the uncertain gap  $[l_1 + 1, u_1]$ . When  $u_1 = +\infty$ , the result is infinite:  $L = [l_1 + l_2, +\infty]$ ,  $\mathbf{p} = \langle p_{1,1} \dots p_{1,l_1} \rangle$ ,  $s = s_1 \sqcup_{\mathcal{B}^\#} \text{SquashExcept}(v_1, [1, l_1]) \sqcup_{\mathcal{B}^\#} \text{Squash}(v_2)$ . (Full case analysis in the appendix.)

### 3.5 Recycling Abstraction

**Recycle $^\#$**  :  $\mathcal{V}_{\mathcal{B}^\#}^\# \times \mathcal{V}_{\mathcal{B}^\#}^\# \rightarrow \mathcal{V}_{\mathcal{B}^\#}^\# \times \mathcal{V}_{\mathcal{B}^\#}^\#$  abstracts the concrete Recycle. Given  $v_1 = ([l_1, u_1], \langle p_{1,1} \dots p_{1,m} \rangle, s_1, a_1)$  and  $v_2 = ([l_2, u_2], \langle p_{2,1} \dots p_{2,n} \rangle, s_2, a_2)$ , let  $[l', u'] = [\max(l_1, l_2), \max(u_1, u_2)]$ . The result is:

$$\text{Recycle}^\#(v_1, v_2) \stackrel{\text{def}}{=} \left( \left( [l', u'], \bigcup_{d=l'}^{\max(m,n)} \rho_f^\#(\mathbf{p}_1, s_1, l_1, d), s'_1, a_1 \right), \left( [l', u'], \bigcup_{d=l'}^{\max(m,n)} \rho_f^\#(\mathbf{p}_2, s_2, l_2, d), s'_2, a_2 \right) \right)$$

where the summary updates to  $s'_i = s_i$  when  $u' \neq +\infty$ , and  $s'_i = s_i \sqcup_{\mathcal{B}^\#} \bigcup_{j=1}^{|p_i|} p_{i,j}$  otherwise, capturing all values that may appear at unknown positions. The helper  $\rho_f^\#$  cycles known content over all possible recycled lengths  $[l', \max(m, n)]$ , joining the results. Internally it uses  $\rho_c^\#$ , which repeats the first  $i$  elements to reach a target length (mirroring the concrete  $\rho_c$ ). (T4) **Recycle $^\#$**  is a sound abstraction of Recycle: for all  $x \in \gamma_{\mathcal{V}_{\mathcal{B}^\#}^\#}(v_1)$  and  $y \in \gamma_{\mathcal{V}_{\mathcal{B}^\#}^\#}(v_2)$ ,  $\text{Recycle}(x, y) \in \gamma_{\mathcal{V}_{\mathcal{B}^\#}^\#}(v'_1) \times \gamma_{\mathcal{V}_{\mathcal{B}^\#}^\#}(v'_2)$  where  $(v'_1, v'_2) = \text{Recycle}^\#(v_1, v_2)$ .

### 3.6 Helper Functions

The selection and update abstractions rely on a small set of utilities (cardinality and enumerability tests on length intervals, content squashing, zero-index adjustment, and value propagation through positions), along with the recycling helpers  $\rho_f^\#$  and  $\rho_c^\#$  introduced above.

### 3.7 Selection Abstraction

The function **Select $^\#$**  :  $\mathcal{V}_{\mathcal{B}^\#}^\# \times \mathcal{V}_{\text{Itv}}^\# \rightarrow \mathcal{V}_{\mathcal{B}^\#}^\#$  abstracts the concrete Select. Since R does not permit mixed positive/negative indices, we apply *abstract filtering* [3]:  $\mathbf{p}_2^+$  retains positions whose concretization is a subset of  $\mathbb{Z}_{\geq 0} \cup \{\text{NA}\}$ ;  $\mathbf{p}_2^-$  retains positions whose concretization is a subset of  $\mathbb{Z}_{\leq 0}$ . Letting  $v_2^+$

and  $v_2^-$  be the filtered selectors:

$$\text{Select}^\#(v_1, v_2) = \text{Select}_{\text{pos}}^\#(v_1, v_2^+) \sqcup_{\mathcal{V}_{\mathcal{B}^\#}^\#} \text{Select}_{\text{neg}}^\#(v_1, v_2^-).$$

The empty selector returns the source unchanged.

**Positive selection.** Abstract element access  $v_1^\#(j)$  retrieves the value at position  $j$ , joining with  $\alpha_{\mathcal{B}^\#}(\text{NA})$  when the position is definitely out of bounds.

If the selector is *finite* ( $u'_2 \neq +\infty$ ): for each enumerable position we extract  $v_1^\#(j)$  joined over all  $j \in \gamma_{\text{Itv}}(p_{2,i}) \setminus \{0\}$ ; non-enumerable positions are summarized into a single joined value. The summary component of the result is  $\perp_{\mathcal{B}^\#}$ .

If the selector is *infinite* ( $u'_2 = +\infty$ ): we proceed similarly, but the result summary captures the joined content of all positions beyond the known content. (T5) **Select $^\#_{\text{pos}}$**  is a sound abstraction of the concrete positive selection.

**Negative selection.** We identify three sets of positions: **MustDeleted** (definitely excluded), **MayDeleted** (possibly excluded, including positions the selector's summary may cover), and **MustNotDeleted** (definitely kept). The result distinguishes three cases based on the adjusted selector: (1) at least one non-enumerable position: the retained content is summarized; (2) infinite selector with all enumerable positions: the result preserves precise position tracking using **MustDeleted** and **MayDeleted** counts; (3) finite selector with all enumerable: similar to case (2) but **MayDeleted** excludes contributions from the selector's summary.

**Logical selection.** A Boolean mask is recycled to match the source length. Each output position is determined by the concretization of the corresponding mask element. Definite-true positions select the element, and possibly-true positions also join with  $\alpha_{\mathcal{B}^\#}(\text{NA})$ . Definite-NA positions yield  $\alpha_{\mathcal{B}^\#}(\text{NA})$ , and definite-false positions are skipped. If the selector is finite, the result is bounded. If it is infinite, the result summary captures values from positions beyond the recycled mask. (Full case analysis in the appendix.)

### 3.8 Update Abstraction

The function **Update $^\#$**  :  $\mathcal{V}_{\mathcal{B}^\#}^\# \times \mathcal{V}_{\text{Itv}}^\# \times \mathcal{V}_{\mathcal{B}^\#}^\# \rightarrow \mathcal{V}_{\mathcal{B}^\#}^\#$  abstracts the concrete Update. As in the selection case, we apply *abstract filtering* to separate positive and negative indices, yielding  $\mathbf{p}_2^+$  and  $\mathbf{p}_2^-$ , and:

$$\text{Update}^\#(v_1, v_2, v_3) = \text{Update}_{\text{pos}}^\#(v_1, v_2^+, v_3) \sqcup_{\mathcal{V}_{\mathcal{B}^\#}^\#} \text{Update}_{\text{neg}}^\#(v_1, v_2^-, v_3).$$

The empty selector updates all positions: we construct a selector  $v_2' = ([l_1, u_1], \langle [1, 1][2, 2] \dots [u_1, u_1] \rangle, s_1, a_1)$  and reduce to  $\text{Update}_{\text{pos}}^\#$ .

**Positive update.** Three cases based on selector length and enumerability. (1) Non-enumerable positions: collapse to  $([l_1, +\infty], \langle v_{\text{all}} \dots v_{\text{all}} \rangle, v_{\text{all}}, a_1)$  where  $v_{\text{all}} = \text{Squash}(v_1) \sqcup_{\mathcal{B}^\#}$

**Table 3.** Representative test cases from the evaluation corpus. The symbol \* denotes a statically unknown value ( $\top_{\mathcal{V}_{\text{Itv}}^\#}$ ).

| Code Snippet                                                                                             | Description                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>r &lt;- if (*) {   c(1, NA, 3) } else {   c(0, NA, 5, NA) }</pre>                                   | Creation - Conditional vector construction.<br>$r : ([3, 4], \langle [0, 1] \alpha_{\text{Itv}}(\text{NA}) [3, 5] \alpha_{\text{Itv}}(\text{NA}) \rangle)$                                                                                                                                                                        |
| <pre>v &lt;- 1 if (*) {   v &lt;- c(1, 2, 3) } else {   v &lt;- c(0, 1) } r &lt;- v + c(9, 10, 11)</pre> | Recycling - Vector addition with uncertain branching.<br>$r : ([3, 3], \langle [9, 10] [11, 12] [11, 14] \rangle)$                                                                                                                                                                                                                |
| <pre>v &lt;- c(1, 2, 3, 4, 5) r &lt;- v[c(1, 3)]</pre>                                                   | Select - Vector subset with multiple indices.<br>$r : ([2, 2], \langle [1, 1] [3, 3] \rangle)$                                                                                                                                                                                                                                    |
| <pre>r &lt;- c(1, 2, 3, 4, 5) r[10] &lt;- 42</pre>                                                       | Update - Vector assignment out of bounds.<br>$r : ([10, 10], \langle [1, 1] \dots [5, 5] \alpha_{\text{Itv}}(\text{NA}) \dots \alpha_{\text{Itv}}(\text{NA}), [42, 42] \rangle)$                                                                                                                                                  |
| <pre>v &lt;- * s &lt;- c(1, 2, 3) x &lt;- s[v] y &lt;- x[c(1, 2)]</pre>                                  | Uncertainty - Top source positive finite selector.<br>$x : ([0, +\infty], \langle \rangle, [1, 3] \sqcup_{\text{Itv}} \alpha_{\text{Itv}}(\text{NA}))$<br>$y : ([2, 2], \langle [1, 3] \sqcup_{\text{Itv}} \alpha_{\text{Itv}}(\text{NA}) [1, 3] \sqcup_{\text{Itv}} \alpha_{\text{Itv}}(\text{NA}) \rangle, \perp_{\text{Itv}})$ |
| <pre>x &lt;- c(1, 2, 3) i &lt;- 1 while(*) {   x &lt;- c(x, i)   i &lt;- i + 1 } r &lt;- x</pre>         | Loop with growing vector triggers widening.<br>$r : ([3, +\infty], \langle [1, 1] [2, 2] [3, 3] \rangle, [1, +\infty] \sqcup_{\text{Itv}} \alpha_{\text{Itv}}(\text{NA}))$                                                                                                                                                        |

Squash( $v_3$ ). (2) Infinite selector with enumerable positions: extend source with  $\alpha_{\mathcal{G}^\#}(\text{NA})$  as needed, apply positionwise strong/weak updates, and set an infinite summary. (3) Finite selector with enumerable positions: similar, but with finite result bounds.

**Negative update.** Reduces to positive update via a constructed selector  $v_s$  that encodes the complement of the excluded positions. (Full construction in the appendix.)

**Logical update.** The mask is recycled to match the source length. Depending on whether the mask is definitely true, possibly true, or definitely false, a strong update, a weak update, or no update is applied per position. (Full case analysis in the appendix.)

### 3.9 Other Operations

**Type coercion.** As in the concrete semantics (Section 2.2), heterogeneous abstract vectors are implicitly coerced to their least upper bound type before evaluation. In our interval instantiation, False maps to  $[0, 0]$  and True to  $[1, 1]$ . Type

information is preserved implicitly by the analysis context, allowing us to distinguish logical from numeric operations without explicitly tracking types in the abstract domain.

**Attributes.**  $\text{setAttr}^\#(v, a) = (L, p, s, a)$  if  $a = \text{AttrEmpty}$ , otherwise  $\top_{\mathcal{V}_B^\#}$ . Special attributes such as `class` and `dim` can redirect an operation to a different S3 method or indexing scheme [18], so none of the attribute-free rules above remain sound once one is present, making a fallback to  $\top_{\mathcal{V}_B^\#}$  the only sound choice. Purely descriptive attributes like `names`, however, do not change an operation's outcome and need not trigger this fallback; separating them precisely would require modeling R's dynamic dispatch, which we leave to future work.

## 4 Evaluation

We implement our abstract domain on top of flowR [17] and instantiate it with intervals as  $\mathcal{D}_{\mathcal{V}_{\text{Itv}}^\#}$ . This proof-of-concept demonstrates feasibility and correctness: since no existing approach formalizes R's full vector semantics [1, 7], we argue correctness from our concrete semantics (Section 2) and systematic comparison against the R interpreter. Each test pairs a small R snippet with an expected abstract value; we verify correctness by comparing computed lengths, contents, and types against the expected ones, and by executing the R code to confirm sound over-approximation.

Our suite covers six categories of vector operations totaling 61 test cases,<sup>1</sup> spanning the core vector semantics; Table 3 presents a representative test for each. *Creation* (14 tests) spans construction from scalars, literals, concatenation, and conditional branches. The representative test highlights a join at a control-flow merge point, where branches produce vectors of different lengths with NA values. *Recycling* (9 tests) exercises element-wise arithmetic with scalar and vector recycling, mismatched lengths, and NA propagation. The representative test adds a length-3 vector to one of length 2 across an uncertain branch. *Select* (12 tests) targets positive, negative, and Boolean indices, out-of-bounds access, and NA selectors. The representative test performs positional selection with concrete indices. *Update* (12 tests) covers subset assignment, negative-index deletion, out-of-bounds extension, and conditional updates. The representative test extends a vector by assigning beyond its length, filling the intermediate positions with NA. *Uncertainty* (8 tests) addresses unbounded vectors arising from abstract interpretation. When selecting from a  $\top$  source, the summary field preserves the value range even though no positions can be tracked, letting subsequent operations recover precision, as the  $y$  result in Table 3 demonstrates. *Loop* (6 tests) focuses on fixpoint computation and widening. The representative test triggers widening on a `while` loop with a growing vector, soundly

<sup>1</sup>Test files and implementation available at <https://github.com/manueldiagostino/flowr>.

over-approximating the length as  $[3, +\infty]$  while preserving known positions. One structural cost is worth noting: out-of-bounds updates materialize intermediate positions as NA, so  $r[10] \leftarrow 42$  on a length-5 vector extends the known content to ten positions, four of them  $\alpha_{\text{Itv}}(\text{NA})$  (Table 3, Update row); in general, an index- $i$  out-of-bounds update materializes  $i$  positions. This growth is bounded inside loops, where widening folds excess positions into the summary once the length becomes unbounded (see the appendix); outside a loop, however, the result length is governed by the largest concrete index assigned.

## 5 Conclusion

In this work, we presented a parametric abstract domain for R vectors together with a core calculus  $\mu R$ . Our abstract domain combines the length, a precise sequence for known positions, a summary for unbounded positions, and an attribute abstraction. We implement a proof-of-concept on top of flowR and evaluate it on a suite of 61 handcrafted test programs that systematically cover the vector features formalized in this paper, showing that the abstraction is sound and precise enough to be useful in practice. Because the domain is parametric in the element abstraction and layers directly onto an existing control-flow graph such as flowR's, it can be adopted by other static analyses for R with comparatively little engineering effort.

Future work includes several directions. We plan to extend  $\mu R$  and the abstract domain to cover more of R's semantics, enabling analysis of a wider range of programs, and to model R's method dispatch systems such as S3 precisely, rather than over-approximating their consequences (Section 2). We also intend to support other vector-derived data structures while preserving shape information. Moreover, our analysis is currently non-relational: it tracks each vector's length independently and cannot relate the lengths of two operands, over-approximating recycling when operands of related but different lengths are aligned; such length relations could be recovered by a reduced product with a relational length domain. Finally, extending the soundness proofs to the remaining operators, which we currently validate only empirically, is left to future work.

## Acknowledgments

This work was supported by the German Research Foundation (DFG): 504226141.

## References

- [1] Martin Bodin, Tomás Díaz, and Éric Tanter. 2018. A trustworthy mechanized formalization of R. In *Proceedings of the 14th ACM SIGPLAN International Symposium on Dynamic Languages, DLS 2018, Boston, MA, USA, November 6, 2018*, Tim Felgentreff (Ed.). ACM, 13–24. doi:10.1145/3276945.3276946
- [2] Giulia Costantini, Pietro Ferrara, and Agostino Cortesi. 2015. A suite of abstract domains for static analysis of string values. *Softw. Pract. Exp.* 45, 2 (2015), 245–287. doi:10.1002/SPE.2218
- [3] P. Cousot and R. Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, 238–252. doi:10.1145/512950.512973
- [4] Patrick Cousot, Radhia Cousot, and Francesco Logozzo. 2011. A parametric segmentation functor for fully automatic and scalable array content analysis. *SIGPLAN Not.* 46, 1 (Jan. 2011), 105–118. doi:10.1145/1925844.1926399
- [5] Oliver Gerstl, Florian Sihler, and Matthias Tichy. 2026. Inferring the Shape of Data Frames in R Programs using Abstract Interpretation. arXiv:2607.03889 [cs.SE] <https://arxiv.org/abs/2607.03889>
- [6] Josselin Giet, Félix Ridoux, and Xavier Rival. 2023. A Product of Shape and Sequence Abstractions. In *Static Analysis - 30th International Symposium, SAS 2023 (Lecture Notes in Computer Science, Vol. 14284)*, Manuel V. Hermenegildo and José F. Morales (Eds.). Springer Nature Switzerland, 310–342. doi:10.1007/978-3-031-44245-2\_15
- [7] Aviral Goel and Jan Vitek. 2019. On the design, implementation, and use of laziness in R. *Proc. ACM Program. Lang.* 3, OOPSLA (2019), 153:1–153:27. doi:10.1145/3360579
- [8] Simon Holm Jensen, Anders Möller, and Peter Thiemann. 2009. Type Analysis for JavaScript. In *Proc. 16th Int. Symp. Static Analysis (SAS) (LNCS, Vol. 5673)*. Springer, 238–255. doi:10.1007/978-3-642-03237-0\_17
- [9] Vineeth Kashyap, Kyle Dewey, Ethan A. Kuefner, John Wagner, Kevin Gibbons, John Sarracino, Ben Wiedermann, and Ben Hardekopf. 2014. JSAI: A Static Analysis Platform for JavaScript. In *Proc. 22nd ACM SIGSOFT Int. Symp. Foundations of Software Engineering (FSE)*. ACM, 121–132. doi:10.1145/2635868.2635904
- [10] Duncan Lang, Roger Peng, Deborah Nolan, and Gabriel Becker. 2018. *CodeDepends*. <https://CRAN.R-project.org/package=CodeDepends>
- [11] Raphaël Monat, Abdelraouf Ouadjaout, and Antoine Miné. 2020. Static Type Analysis by Abstract Interpretation of Python Programs. In *34th European Conference on Object-Oriented Programming (ECOOP) (LIPIcs, Vol. 166)*. Schloss Dagstuhl, 17:1–17:29. doi:10.4230/LIPIcs.ECOOP.2020.17
- [12] Floréal Morandat, Brandon Hill, Leo Osvald, and Jan Vitek. 2012. Evaluating the Design of the R Language: Objects and Functions for Data Analysis. In *26th European Conference on Object-Oriented Programming (ECOOP) (LNCS, Vol. 7313)*. Springer, 104–131. doi:10.1007/978-3-642-31057-7\_6
- [13] Luca Negrini, Guruprereana Shabadi, and Caterina Urban. 2023. Static Analysis of Data Transformations in Jupyter Notebooks. In *Proceedings of the 12th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis, SOAP 2023, Orlando, FL, USA, 17 June 2023*, Pietro Ferrara and Liana Hadarean (Eds.). ACM, 8–13. doi:10.1145/3589250.3596145
- [14] R Core Team. 2026. *R Language Definition*. R Foundation for Statistical Computing. <https://cran.r-project.org/doc/manuals/r-release/R-lang.html>
- [15] Rathijit Sen, Jianqiao Zhu, Jignesh M. Patel, and Somesh Jha. 2017. ROSA: R Optimizations with Static Analysis. In *RIOT 2017*. arXiv. <https://www.microsoft.com/en-us/research/publication/rosa-r-optimizations-with-static-analysis/>
- [16] Florian Sihler, Lukas Pietzschmann, Raphael Straub, Matthias Tichy, Andor Diera, and Abdelhalim Dahou. 2024. On the Anatomy of Real-World R Code for Static Analysis. In *Proc. 21st Int. Conf. Mining Software Repositories (MSR)*. ACM, 619–630. doi:10.1145/3643991.3644911
- [17] Florian Sihler and Matthias Tichy. 2025. Statically Analyzing the Dataflow of R Programs. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 309 (2025), 29 pages. doi:10.1145/3763087
- [18] Hadley Wickham. 2019. *Advanced R* (2nd ed.). CRC Press, Boca Raton, FL. <https://adv-r.hadley.nz/vectors-chap.html>

Received 2026-07-18; accepted 2026-08-17